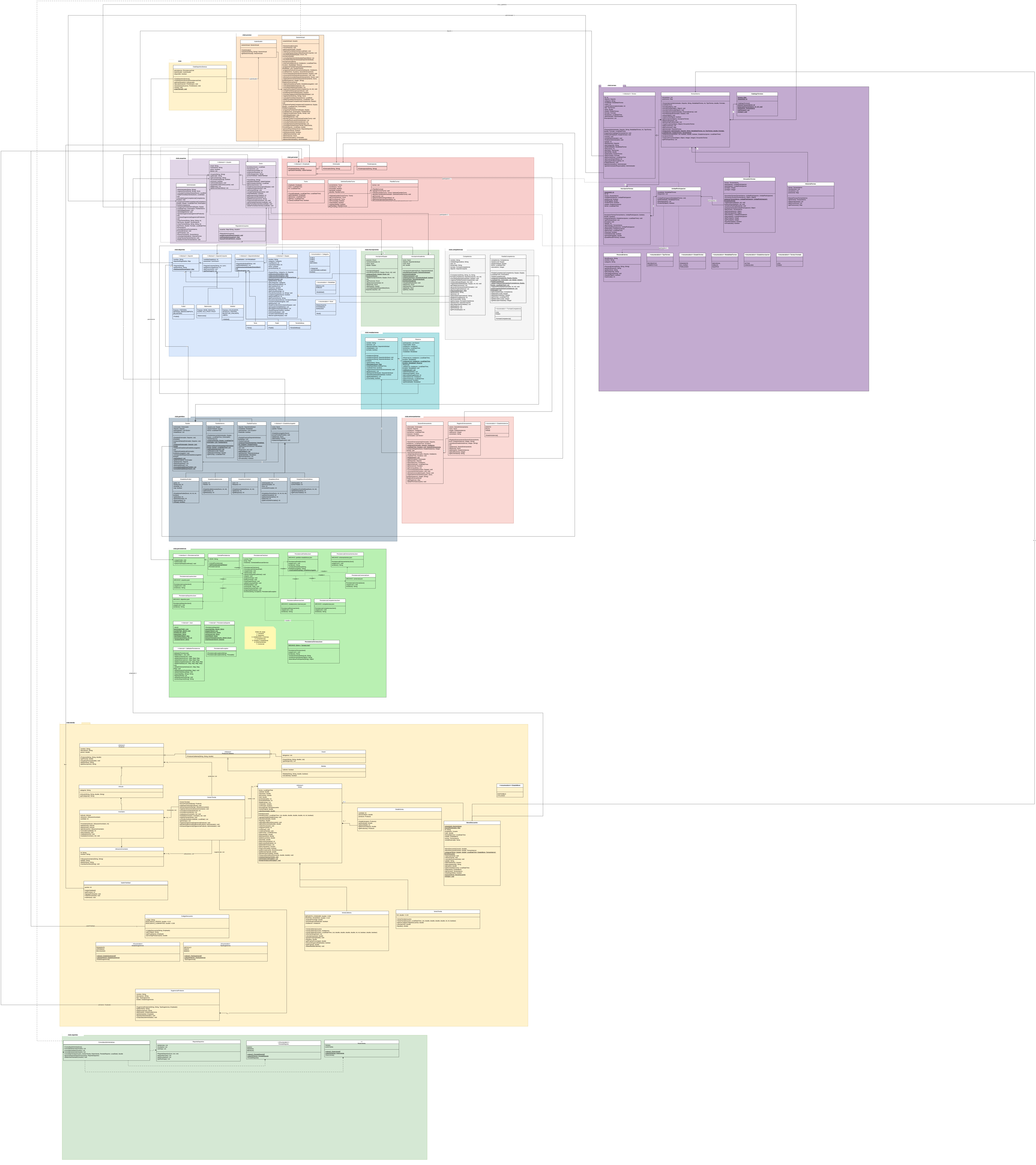
club
ClubDeportivoSeneca
-persistencia: IPersistenciaClub
-autenticador: Autenticador
-disponible: boolean
+ClubDeportivoSeneca()
+ClubDeportivoSeneca(IPersistenciaClub)
+getAutenticador(): Autenticador
+ejecutar(Scanner, PrintStream): void
-iniciarSesion(Scanner, PrintStream): void
+close(): void
+main(String[]): void
-autenticador
-persistencia
club.acceso
Autenticador
-sesionActual: SesionActual
+Autenticador()
+autenticar(String, String): SesionActual
+getSesionActual(): SesionActual
SesionActual
-usuarioActual: Usuario
+SesionActual(Usuario)
+cerrarSesion(): void
+getUsuarioActual(): Usuario
+registrarFechaNacimiento(LocalDate): void
+consultarEquiposDisponibles(DeporteConjunto): List
+inscribirEnEquipo(Equipo, Enum, int): InscripcionEquipo
+consultarDeportesIndividualesDisponibles(): List
+inscribirEnDeporteIndividual(DeporteIndividual): InscripcionAcademia
+reservarInstalacion(List, Instalacion, LocalDateTime, Duration, Modalidad): Reserva
+programarPartidoPractica(DeporteIndividual, Modalidad, List): PartidoPractica
+programarSesionEntrenamiento(Deporte, Instalacion, LocalDateTime, Duration): SesionEntrenamiento
+convocarEquipo(SesionEntrenamiento, Equipo): void
+convocarSocios(SesionEntrenamiento, List): void
+eliminarConvocado(SesionEntrenamiento, Socio): void
+registrarEntrenamiento(SesionEntrenamiento, Socio, EstadoAsistencia, Integer, String): RegistroEntrenamiento
+registrarEstadistica(Partido, EstadisticaJugador): void
+consultarEstadisticas(Socio): List
+consultarEstadisticas(Partido): List
+registrarResultado(PartidoCompetencia, int, int): void
+cumplirPartidoSuspension(Socio): void
+puedeDisputarPartido(Equipo): boolean
+consultarJugadoresNoDisponibles(Equipo): List
+registrarSocio(String, String): Socio
+consultarFechaNacimiento(Socio): LocalDate
+editarFechaNacimiento(Socio, LocalDate): void
+inscribirEquipoCompetencia(Competencia, Equipo): void
+programarPartidoCompetencia(Competencia, Equipo, Equipo, LocalDateTime, Entrenador): PartidoCompetencia
+programarPartidoInterno(Equipo, Equipo, LocalDateTime, Entrenador): PartidoInterno
+registrarVenta(GestorTienda, Venta): void
+darDeBaja(Equipo): void
+darDeBaja(Socio): void
+aprobarCambioTurno(SolicitudCambioTurno): void
+consultarReservasAdministrativas(): List
+consultarPartidosAdministrativos(): List
+consultarSesionesAdministrativas(): List
+consolidarVentas(GestorTienda, RubroVenta, PeriodoReporte, LocalDate): double
+generarReporteDeportivo(): ReporteDeportivo
+haySesionAtiva(): boolean
+haySesionActiva(): boolean
-validarSesionActiva(): void
-obtenerSocio(): Socio
-obtenerEntrenador(): Entrenador
-obtenerAdministrador(): Administrador
club.usuarios
<<Abstract>> Usuario
-email: String
-contrasenia: String
-activo: boolean
-bonos: List
+Usuario(String, String)
+getEmail(): String
+getContrasenia(): String
+isContrasenia(String): boolean
+isActivo(): boolean
+setActivo(boolean): void
+guardarBono(BonoDescuento): void
+getBonos(): List
+getBonosDisponibles(): List
Socio
-fechaNacimiento: LocalDate
-inscripciones: List
-suspensionesTotales: int
-suspensionRestante: int
-motivoSuspension: String
-puntosFidelidad: SaldoFidelidad
+Socio(String, String)
+getSaldoFidelidad(): SaldoFidelidad
+getFechaNacimiento(): LocalDate
+getInscripciones(): List
+anadirInscripcion(InscripcionEquipo): void
+registrarSuspension(int): void
+sancionar(String): void
+cumplirPartidoSuspension(): void
+estaHabilitado(): boolean
+getSuspensionesTotales(): int
+getSuspensionRestante(): int
+getMotivoSuspension(): String
+restaurarSuspensiones(int, int): void
+setFechaNacimiento(LocalDate): void
editarFechaNacimiento(LocalDate): void
-validarFechaNacimiento(LocalDate): void
Administrador
+Administrador(String, String)
+registrarSocio(String, String): Socio
+consultarFechaNacimiento(Socio): LocalDat
+editarFechaNacimiento(Socio, LocalDate): void
+inscribirEquipoCompetencia(Competencia, Equipo): void
+programarPartidoCompetencia(Competencia, Equipo, Equipo, LocalDateTime, Entrenador): PartidoCompetencia
+programarPartidoInterno(Equipo, Equipo, LocalDateTime, Entrenador): PartidoInterno
+darDeBaja(Equipo): void
+darDeBaja(Socio): void
+aprobarSugerencia(SugerenciaProducto): void
+rechazarSugerencia(SugerenciaProducto): void
+crearTorneo(String, String, String, int, TipoTorneo, double): TorneoInterno
+crearTorneo(String, String, String, int, TipoTorneo, double, Formato, LocalDateTime TorneoInterno
+inscribirExterno(TorneoInterno, PersonaExterna): void
+getTorneos(): List
+buscarTorneo(int): TorneoInterno
+consultarHistorial(int): HistorialTorneo
+iniciarTorneo(TorneoInterno): void
+finalizarTorneo(TorneoInterno): void
RepositorioUsuarios
-usuarios: Map<String, Usuario>
+RepositorioUsuarios()
+anadirUsuarioARepo(Usuario): void
+getRepositorioUsuarios(): Map
+buscarPorEmail(String): Usuario
club.personal
<<Abstract>> Empleado
+Empleado(String, String)
+getSaldoFidelidad(): SaldoFidelidad
Entrenador
+Entrenador(String, String)
Fisioterapeuta
+Fisioterapeuta(String, String)
Turno
-empleado: Empleado
-inicio: LocalDateTime
-fin: LocalDateTime
+Turno(Empleado, LocalDateTime, LocalDateTime)
+getEmpleado(): Empleado
+getInicio(): LocalDateTime
+getFin(): LocalDateTime
+cubre(LocalDateTime): boolean
SolicitudCambioTurno
-turnoAnterior: Turno
-turnoNuevo: Turno
-procesada: boolean
-aprobada: boolean
-plantilla: PlantillaTurnos
+SolicitudCambioTurno(Turno, Turno)
+getTurnoAnterior(): Turno
+getTurnoNuevo(): Turno
+isProcesada(): boolean
+isAprobada(): boolean
+estaAprobada(): boolean
getPlantilla(): PlantillaTurnos
PlantillaTurnos
-turnos: List
+PlantillaTurnos()
+agregarTurno(Turno): void
+solicitarCambio(Turno, Turno): SolicitudCambioTurno
+aprobarCambio(Administrador, SolicitudCambioTurno): void
+getTurnos(): List
+getEntrenadoresDisponibles(): int
+getFisioterapeutasDisponibles(): int
participante 0..*
club.deportes
<<Abstract>> Deporte
-nombre: String
-deportesRegistrados: Map
+Deporte(String)
+getNombre(): String
+getDeportesRegistrados(): Map
<<Abstract>> DeporteConjunto
-cantidadJugadores: int
-posiciones: List<Enum>
+DeporteConjunto(String, int, List>)
+getMinimoJugadores(): int
+getPosiciones(): List>
+getRanking(int): List
<<Abstract>> DeporteIndividual
-modalidades: List<Modalidad>
+DeporteIndividual(String, List)
+getModalidades(): List
+getDeportesIndividualesDisponibles(): List
+getRanking(int): List
<<Abstract>> Equipo
-nombre: String
-categoria: Categoria
-cupoMaximo: int
-cuposDisponibles: int
-activo: boolean
-puntosRanking: int
+Equipo(String, Categoria, int, Deporte)
+getEquiposDisponibles(): Map
+getEquiposRegistrados(): Map
+getEquiposDisponiblesPorCategoria(Socio, DeporteConjunto): List
+getNombre(): String
+getCategoria(): Categoria
+getCuposDisponibles(): int
+getCupoMaximo(): int
+isActivo(): boolean
+getDeporte(): Deporte
+getPuntosRanking(): int
+inscribirSocio(Socio, int, String): void
+contieneJugador(Socio): boolean
+getDorsal(Socio): int
+getPosicion(Socio): String
-buscarInscripcion(Socio): InscripcionEquipo
+sumarPuntosRanking(int): void
+getNomina(): List
+anadirInscripcion(InscripcionEquipo): void
+getJugadoresHabilitados(): List
+getJugadoresNoDisponibles(): List
+puedeDisputarPartido(): boolean
+setActivo(boolean): void
+ocuparCupoEnEquipo(): void
+liberarCupoEnEquipo(): void
<<enumeration>> Categoria
SUB13
SUB15
SUB17
MAYORES
-Categoria()
+admiteEdad(LocalDate): boolean
<<enumeration>> Modalidad
SENCILLOS
DOBLES
-Modalidad()
<<enumeration>> Nivel
PRINCIPIANTE
INTERMEDIO
AVANZADO
-Nivel()
Futbol
Posicion: PORTERO, DEFENSA, MEDIOCAMPISTA, DELANTERO
+Futbol()
Baloncesto
Posicion: BASE, ESCOLTA, ALERO, ALA_PIVOT, PIVOT
+Baloncesto()
Voleibol
Posicion: COLOCADOR, OPUESTO, CENTRAL, RECEPTOR_ATACANTE, LIBERO
+Voleibol()
Tenis
+Tenis()
Padel
+Padel()
TenisDeMesa
+TenisDeMesa()
2..4
1
0..*
club.Inscripciones
InscripcionEquipo
-posicion: Enum
-numeroCamiseta: int
-socio: Socio
-equipo: Equipo
+InscripcionEquipo()
+inscribirSocio(Socio, Equipo, Enum, int): void
+restaurar(Socio, Equipo, Enum, int): InscripcionEquipo
-validarInscripcion(Socio, Equipo, Enum, int): void
+getPosicion(): Enum
+getNumeroCamiseta(): int
+getSocio(): Socio
+getEquipo(): Equipo
+consultarEquiposDisponibles(Socio, DeporteConjunto): List
InscripcionAcademia
-socio: Socio
-deporte: DeporteIndividual
-nivel: Nivel
-elo: double
-InscripcionAcademia(Socio, DeporteIndividual)
+inscribirSocio(Usuario, DeporteIndividual): InscripcionAcademia
+getInscripciones(): List
+estaInscrito(Socio, DeporteIndividual): boolean
+buscar(Socio, DeporteIndividual): InscripcionAcademia
+actualizarElo(double): void
+getSocio(): Socio
+getDeporte(): DeporteIndividual
+getNivel(): Nivel
+getElo(): double
club.competencias
Competencia
-nombre: String
-entidadOrganizadora: String
-anio: int
-temporada: String
-formato: FormatoCompetencia
-deporte: DeporteConjunto
+Competencia(String, String, int, String, FormatoCompetencia, DeporteConjunto, List, int)
+inscribirEquipo(Administrador, Equipo): void
+registrarPartido(PartidoCompetencia): void
+restaurarEquipo(Equipo, int): void
+registrarResultado(Equipo, Equipo, int, int): void
+getTablaPosiciones(): List
+generarEnfrentamientos(Entrenador): List
+getRankingEstadistica(): List
+getCompetencias(): Map
+getNombre(): String
+getAnio(): int
+getEntidadOrganizadora(): String
+getMinimoJugadores(): int
+getTemporada(): String
+getFormato(): FormatoCompetencia
+getDeporte(): DeporteConjunto
+getCategoriasPermitidas(): List
+getEquipos(): List
+getPartidos(): List
+getPuntos(Equipo): int
PartidoCompetencia
-competencia: Competencia
-equipoLocal: Equipo
-equipoVisitante: Equipo
-fecha: LocalDateTime
-marcadores: Integer
+PartidoCompetencia(Competencia, Equipo, Equipo, LocalDateTime, Entrenador)
-PartidoCompetencia()
+restaurar(Competencia, Equipo, Equipo, LocalDateTime, Entrenador, List, Integer, Integer): PartidoCompetencia
-validarYObtenerParticipantes(Competencia, Equipo, Equipo, LocalDateTime): List
+registrarResultado(Entrenador, int, int): void
crearEmparejamientos(Competencia, List, Entrenador): List
+getCompetencia(): Competencia
+getEquipoLocal(): Equipo
+getEquipoVisitante(): Equipo
+getFecha(): LocalDateTime
+getMarcadorLocal(): Integer
+getMarcadorVisitante(): Integer
<<enumeration>> FormatoCompetencia
LIGA
FASES
-FormatoCompetencia()
club.instalaciones
Instalacion
-nombre: String
-sesiones: List
-deporteIndividual: DeporteIndividual
-modalidades: List
-techada: boolean
+Instalacion(String)
+Instalacion(String, DeporteIndividual, List)
+Instalacion(String, DeporteIndividual, List, boolean)
+getNombre(): String
+getInstalaciones(): Map
+estaDisponible(LocalDateTime, LocalDateTime): boolean
+registrarSesion(SesionEntrenamiento): void
+getSesiones(): List
+getDeporteIndividual(): DeporteIndividual
+admiteModalidad(Modalidad): boolean
+getModalidades(): List
+isTechada(): boolean
Reserva
-participantes: List<Socio>
-responsable: Socio
-instalacion: Instalacion
-fechaHora: LocalDateTime
-duracion: Duration
-modalidad: Modalidad
+Reserva(List, Instalacion, LocalDateTime, Duration, Modalidad)
+restaurar(List, Instalacion, LocalDateTime, Duration, Modalidad): Reserva
-Reserva()
-validar(List, Instalacion, LocalDateTime, Duration, Modalidad): void
+getReservas(): List
+getParticipantes(): List
+getResponsable(): Socio
+getCantidadJugadores(): int
+getInstalacion(): Instalacion
+getFechaHora(): LocalDateTime
+getDuracion(): Duration
+getModalidad(): Modalidad
club.partidos
Partido
-entrenador: Entrenador
-deporte: Deporte
-participantes: List<Socio>
-estadisticas: List
+Partido(Entrenador, Deporte, List)
#Partido()
#restaurarDatos(Entrenador, Deporte, List): void
+restaurar(Entrenador, Deporte, List): Partido
+restaurarEstadistica(EstadisticaJugador): void
+registrarEstadistica(Entrenador, EstadisticaJugador): void
-correspondeAlDeporte(EstadisticaJugador): boolean
+getPartidos(): List
+getEntrenador(): Entrenador
+getDeporte(): Deporte
+getParticipantes(): List
+getEstadisticas(): List
+consultarEstadisticas(Partido): List
+consultarEstadisticas(Socio): List
PartidoInterno
-equipoLocal: Equipo
-equipoVisitante: Equipo
-fecha: LocalDateTime
+PartidoInterno(Administrador, Equipo, Equipo, LocalDateTime, Entrenador)
-PartidoInterno()
+restaurar(Equipo, Equipo, LocalDateTim Entrenador, List): PartidoInterno
-validarYObtenerParticipantes(Administrac Equipo, Equipo, LocalDateTime): List
+getPartidosInternos(): List
+getEquipoLocal(): Equipo
+getEquipoVisitante(): Equipo
+getFecha(): LocalDateTime
PartidoPractica
-deporte: DeporteIndividual
-modalidad: Modalidad
-participantes: List<Usuario>
-finalizado: boolean
+PartidoPractica(DeporteIndividual, Modalidad, List)
-PartidoPractica()
+restaurar(DeporteIndividual, Modalidad, List, boolean): PartidoPractica
-validar(DeporteIndividual, Modalidad, List): void
+finalizar(int, int): void
+getPartidos(): List
+getDeporte(): DeporteIndividual
+getModalidad(): Modalidad
+getParticipantes(): List
+isFinalizado(): boolean
<<Abstract>> EstadisticaJugador
-socio: Socio
-partido: Partido
+EstadisticaJugador(Socio)
asociarPartido(Partido): void
+getSocio(): Socio
+getPartido(): Partido
#validarNoNegativo(int): void
EstadisticaFutbol
-goles: int
-asistencias: int
-amarillas: int
-roja: boolean
+EstadisticaFutbol(Socio, int, int, int, boolean)
+getGoles(): int
+getAsistencias(): int
+getAmarillas(): int
+isRoja(): boolean
EstadisticaBaloncesto
-puntos: int
-rebotes: int
+EstadisticaBaloncesto(Socio, int, int)
+getPuntos(): int
+getRebotes(): int
EstadisticaVoleibol
-aces: int
-bloqueos: int
+EstadisticaVoleibol(Socio, int, int)
+getAces(): int
+getBloqueos(): int
EstadisticaTenis
-setsGanados: int
-gamesGanados: int
-aces: int
-erroresNoForzados: int
+EstadisticaTenis(Socio, int, int, int, int)
+getSetsGanados(): int
+getGamesGanados(): int
+getAces(): int
+getErroresNoForzados(): int
EstadisticaTenisDeMesa
-setsGanados: int
-puntosTotales: int
+EstadisticaTenisDeMesa(Socio, int, int)
+getSetsGanados(): int
+getPuntosTotales(): int
club.entrenamientos
SesionEntrenamiento
-entrenador: Entrenador
-deporte: Deporte
-instalacion: Instalacion
-fechaHora: LocalDateTime
-duracion: Duration
-convocados: List<Socio>
+SesionEntrenamiento(Entrenador, Deporte, Instalacion, LocalDateTime, Duration)
+restaurar(Entrenador, Deporte, Instalacion, LocalDateTime, Duration, List): SesionEntrenamiento
+restaurarRegistro(Socio, EstadoAsistencia, Integer, String): void
-SesionEntrenamiento()
-validarDatos(Entrenador, Deporte, Instalacion, LocalDateTime, Duration): void
+getSesiones(): List
+getEntrenador(): Entrenador
+getDeporte(): Deporte
+getInstalacion(): Instalacion
+getFechaHora(): LocalDateTime
+getDuracion(): Duration
+getConvocados(): List
+convocarEquipo(Usuario, Equipo): void
+convocarSocios(Usuario, List): void
+eliminarConvocado(Usuario, Socio): void
+registrarEntrenamiento(Usuario, Socio, EstadoAsistencia, Integer, String): RegistroEntrenamiento
+getRegistros(): Map
-validarPermiso(Usuario): void
RegistroEntrenamiento
-sesion: SesionEntrenamiento
-socio: Socio
-estado: EstadoAsistencia
-calificacion: Integer
-comentario: String
+RegistroEntrenamiento(SesionEntrenamiento, Socio, EstadoAsistencia, Integer, String)
+actualizar(EstadoAsistencia, Integer, String): void
+getSesion(): SesionEntrenamiento
+getSocio(): Socio
+getEstado(): EstadoAsistencia
+getCalificacion(): Integer
+getComentario(): String
<<enumeration>> EstadoAsistencia
ASISTIO
FALTO
TARDE
-EstadoAsistencia()
club.torneo
<<Abstract>> Torneo
-id: int
-deporte: Deporte
-categoria: String
-modalidad: ModalidadTorneo
-cupos: int
-cuposPreferencialesIniciales: int
-tipo: TipoTorneo
-tarifa: double
-estado: EstadoTorneo
-formato: Formato
-fechaInicio: LocalDateTime
-administrador: Administrador
#inscripciones: List
#Torneo(Administrador, Deporte, String, ModalidadTorneo, int, TipoTorneo, double, Formato, LocalDateTime)
#registrarEnCatalogo(TorneoInterno): void
#restaurarEstado(int, EstadoTorneo): void
+iniciar(): void
+iniciar(Administrador): void
+finalizar(Administrador): void
+getId(): int
+getDeporte(): Deporte
+getCategoria(): String
+getModalidad(): ModalidadTorneo
+getCupos(): int
+getTipo(): TipoTorneo
+getTarifa(): double
+getEstado(): EstadoTorneo
+getFormato(): Formato
+getFechaInicio(): LocalDateTime
+getCuposDisponibles(): int
+getCuposPreferenciales(): int
#validarAbierto(): void
#getAdministrador(): Administrador
+getAdministradorResponsable(): Administrador
TorneoInterno
-encuentros: List
-posiciones: Map
+TorneoInterno(Administrador, Deporte, String, ModalidadTorneo, int, TipoTorneo, double, Formato, LocalDateTime)
+inscribir(Object): void
+inscribirPareja(Object, Object): void
+inscribirPreferencial(Object): void
+inscribirExterno(Administrador, PersonaExterna): void
+inscribirEquipo(Entrenador, Equipo): void
+retirar(Object): void
+estaInscrito(Object): boolean
+getInscripcion(Object): InscripcionTorneo
+getInscripciones(): List
+getRankingPreferencial(): List
+crearEncuentro(Object, Object): EncuentroTorneo
+getEncuentros(): List
+getPosiciones(): Map
+getHistorial(): HistorialTorneo
+restaurar(int, Administrador, Deporte, String, ModalidadTorneo, int, TipoTorneo, double, Formato, LocalDateTime, EstadoTorneo): TorneoInterno
+restaurarInscripcion(int, List, boolean, double, boolean, EstadoInscripcion, LocalDateTime): InscripcionTorneo
+restaurarEncuentro(Object, Object, Integer, Integer): EncuentroTorneo
+getParticipantes(): List
CatalogoTorneos
-torneos: Map
-siguienteId: int
-CatalogoTorneos()
registrar(TorneoInterno): int
restaurarId(TorneoInterno, int, int): void
+buscar(int): TorneoInterno
+getTorneos(): List
+limpiar(): void
0..*
1
InscripcionTorneo
-siguienteId: int
-id: int
-torneo: TorneoInterno
-unidadParticipacion: UnidadParticipacion
-preferencial: boolean
-tarifaPagada: double
-recibePremio: boolean
-estado: EstadoInscripcion
-fecha: LocalDateTime
InscripcionTorneo(TorneoInterno, UnidadParticipacion, boolean, double, boolean)
restaurarDatos(int, EstadoInscripcion, LocalDateTime): void
+getParticipantes(): List
cancelar(): void
+getId(): int
+getTorneo(): TorneoInterno
+getUnidadParticipacion(): UnidadParticipacion
+getEstado(): EstadoInscripcion
+getFecha(): LocalDateTime
+isActiva(): boolean
+esPreferencial(): boolean
+getTarifaPagada(): double
+puedeRecibirPremio(): boolean
UnidadParticipacion
-modalidad: ModalidadTorneo
-integrantes: List
+UnidadParticipacion(ModalidadTorneo, List)
+getModalidad(): ModalidadTorneo
+getIntegrantes(): List
+contiene(Object): boolean
EncuentroTorneo
-torneo: TorneoInterno
-participante1: UnidadParticipacion
-participante2: UnidadParticipacion
-puntaje1: Integer
-puntaje2: Integer
-EncuentroTorneo(TorneoInterno, UnidadParticipacion, UnidadParticipacion)
+EncuentroTorneo(TorneoInterno, Object, Object)
restaurar(TorneoInterno, UnidadParticipacion, UnidadParticipacion): EncuentroTorneo
+registrarResultado(int, int): void
restaurarResultado(int, int): void
-actualizarRankingEquipos(): void
-participanteSimple(UnidadParticipacion): Object
+getTorneo(): TorneoInterno
+getParticipante1(): Object
+getParticipante2(): Object
+getUnidad1(): UnidadParticipacion
+getUnidad2(): UnidadParticipacion
+getPuntaje1(): Integer
+getPuntaje2(): Integer
+tieneResultado(): boolean
+isFinalizado(): boolean
participantes 2
HistorialTorneo
-torneo: TorneoInterno
-inscripciones: List
-encuentros: List
-posiciones: Map
HistorialTorneo(TorneoInterno)
+getTorneo(): TorneoInterno
+getInscripciones(): List
+getEncuentros(): List
+getPosiciones(): Map
participante 0..*
PersonaExterna
-identificacion: String
-nombre: String
-categoria: String
+PersonaExterna(String, String, String)
+getIdentificacion(): String
+getNombre(): String
+getCategoria(): String
+setCategoria(String): void
+equals(Object): boolean
+hashCode(): int
<<enumeration>> TipoTorneo
RECREATIVO
COMPETITIVO
<<enumeration>> EstadoTorneo
PENDIENTE
EN_CURSO
FINALIZADO
<<enumeration>> ModalidadTorneo
INDIVIDUAL
DOBLES
EQUIPOS
<<enumeration>> EstadoInscripcion
ACTIVA
CANCELADA
<<enumeration>> Torneo.Formato
LIGA
FASES
club.persistencia
<<interface>> IPersistenciaClub
+cargarClub(): void
+salvarClub(): void
+salvarClubHastaConfirmar(): void
CentralPersistencia
+JSON: String
-CentralPersistencia()
+getPersistenciaClub(String): IPersistenciaClub
PersistenciaClubJson
-current: Path
-temp: Path
-reintentos: ScheduledExecutorService
+PersistenciaClubJson()
+PersistenciaClubJson(Path)
+cargarClub(): void
+salvarClub(): void
+salvarClubHastaConfirmar(): void
-cargar(): void
-prepararTemp(): void
-publicarTemp(): void
-crearEstadoVacio(): void
-validarConjunto(Path): void
-limpiarEstado(): void
-mover(Path, Path): void
-limpiarDirectorio(Path): void
-programarReintento(): void
-envolver(String, Exception): PersistenciaException
<<builds>>
PersistenciaPartidosJson
ARCHIVO: partidos-estadisticas.json
PersistenciaPartidosJson()
cargar(List>): void
serializar(): String
-serializarEstadistica(String, EstadisticaJugador): String
-crearEstadistica(Map): EstadisticaJugador
PersistenciaEntrenamientosJson
ARCHIVO: entrenamientos.json
PersistenciaEntrenamientosJson()
cargar(List>): void
serializar(): String
PersistenciaComercialJson
ARCHIVO: comercial.json
PersistenciaComercialJson()
cargar(List>): void
serializar(): String
PersistenciaUsuariosJson
ARCHIVO: usuarios.json
PersistenciaUsuariosJson()
cargar(List>): void
serializar(): String
PersistenciaDeportivoJson
ARCHIVO: deportivo.json
PersistenciaDeportivoJson()
cargar(List>): void
serializar(): String
PersistenciaReservasJson
ARCHIVO: instalaciones-reservas.json
PersistenciaReservasJson()
cargar(List>): void
serializar(): String
PersistenciaCompetenciasJson
ARCHIVO: competencias.json
PersistenciaCompetenciasJson()
cargar(List>): void
serializar(): String
<<internal>> Json
-Json()
leerArreglo(Path): List>
escribir(Path, String): void
arreglo(List): String
objeto(Map): String
-leerObjeto(String): Map
-buscarCierre(String, int): int
- escapar(String): String
<<internal>> PersistenciaSoporte
-PersistenciaSoporte()
requerido(Map, String): String
separar(String): List
unirEnums(List>): String
unirSocios(List): String
socio(String): Socio
posicion(DeporteConjunto, String): Enum
crearDeporte(String): Deporte
Orden de carga
1. Usuarios
2. Deportivo
3. Instalaciones y reservas
4. Competencias
5. Partidos y estadísticas
6. Entrenamientos
7. Comercial
PersistenciaTorneosJson
ARCHIVO: String = "torneos.json"
PersistenciaTorneosJson()
cargar(List>): void
serializar(): String
-serializarParticipantes(List): String
-serializarParticipante(Object): String
-deserializarParticipante(String): Object
<<internal>> ValidadorPersistencia
ValidadorPersistencia()
validar(Map>>, int): void
-validarUsuarios(List>): Map
-validarDeportivo(List>, Map, Map): Map
-validarReservas(List>, Map, Map): Map
-validarCompetencias(List>, Map, Map): Map
-validarPartidos(List>, Map, Map, Map): void
-validarEntrenamientos(List>, Map, Map, Map): void
-validarEquiposPartido(Map, Map): void
-validarEstadistica(Map): void
-requerido(Map, String): String
-separar(String): List
-validarBooleano(String): void
-nombreDeporte(String): String
PersistenciaException
+PersistenciaException(String)
+PersistenciaException(String, Throwable)
propietario 1
club.tienda
<Abstract> Producto
-nombre: String
-descripcion: String
-precio: double
+Producto(String, String, double)
+getPrecio(): double
+actualizarPrecio(double): void
+getNombre(): String
+getDescripcion(): String
<Abstract> ProductoCafeteria
+ProductoCafeteria(String, String, double)
Snack
-alergenos: List
+Snack(String, String, double, List)
+getAlergenos(): List
Bebida
-caliente: boolean
+Bebida(String, String, double, boolean)
+esCaliente(): boolean
-productos: List
0...*
Articulo
-categoria: String
+Articulo(String, String, double, String)
+getCategoria(): String
GestorTienda
+GestorTienda()
+buscarProducto(String): Producto
+registrarProducto(Producto): void
+buscarUbicacion(String): UbicacionInventario
+registrarUbicacion(UbicacionInventario): void
+consultarInventario(Articulo): int
+consultarInventarios(): List
+reabastecer(Inventario, int): void
+trasladar(Inventario, Inventario, int): void
+registrarVenta(Venta): void
+consultarVentas(LocalDate, LocalDate): List
+getVentas(): List
+registrarSugerencia(SugerenciaProducto): void
+aprobarSugerencia(SugerenciaProducto, Administrador): void
+rechazarSugerencia(SugerenciaProducto, Administrador): void
-inventarios: List
-ventas: List
-ubicaciones: List
-sugerencias: List
Inventario
-articulo: Articulo
-ubicacion: UbicacionInventario
-cantidad: int
+Inventario(Articulo, UbicacionInventario, int)
+getArtiuclo(): Articulo
+getArticulo(): Articulo
+getUbicacion(): UbicacionInventario
+consultarCantidad(): int
+descontar(int): void
+reabastecer(int): void
+trasladar(Inventario, int): void
UbicacionInventario
-id: String
-nombre: String
+UbicacionInventario(String, String)
+getId(): String
+getNombre(): String
+cambiarNombre(String): void
<Abstract> Venta
-fecha: LocalDateTime
-subtotal: double
-impuestos: double
-descuentos: double
-total: double
-puntosGanados: int
-puntosRedimidos: int
-detalleVentas: List
-comprador: Usuario
-confirmada: boolean
-bonoAplicado: BonoDescuento
-montoOriginal: double
-valorDescontado: double
#Venta(Usuario)
#Venta(Usuario, LocalDateTime, List, double, double, double, double, int, int, boolean)
+agregarDetalle(DetalleVenta): void
#calcularSubtotal(): double
+liquidar(): double
+aplicarBono(BonoDescuento): void
+aplicarDescuento(double): double
+redimirPuntos(int): void
+asignarPuntos(): int
+confirmar(): void
+confirmarVenta(): void
+getFecha(): LocalDateTime
+getSubtotal(): double
+getImpuestos(): double
+getDescuentos(): double
+getTotal(): double
+getPuntosGanados(): int
+getPuntosRedimidos(): int
+getDetalleVentas(): List
+getComprador(): Usuario
+estaConfirmada(): boolean
+getBonoAplicado(): BonoDescuento
+getMontoOriginal(): double
+getValorDescontado(): double
+restaurarBono(BonoDescuento, double, double): void
+registrarHistorica(Venta): void
+getVentasConfirmadas(): List
+limpiarVentasConfirmadas(): void
DetalleVenta
-cantidad: int
-precioUnitario: double
-producto: Producto
+DetalleVenta(int, Producto)
+getSubtotal(): double
+getCantidad(): int
+getPrecioUnitario(): double
+getProducto(): Producto
0..1
<<enumeration>> EstadoBono
DISPONIBLE
UTILIZADO
BonoDescuento
-siguienteId: AtomicInteger
-bonosRegistrados: Map
-id: String
-propietario: Usuario
-valor: double
-fechaObtencion: LocalDateTime
-estado: EstadoBono
-torneo: TorneoInterno
-ventaReservada: Venta
+BonoDescuento(Usuario, double)
+BonoDescuento(Usuario, double, TorneoInterno)
+restaurar(String, Usuario, double, LocalDateTime, EstadoBono, TorneoInterno): BonoDescuento
+reservar(Venta): void
+utilizar(Venta): void
+cancelarReserva(Venta): void
+getId(): String
+getPropietario(): Usuario
+getPropietarioId(): String
+getValor(): double
+getFechaObtencion(): LocalDateTime
+getEstado(): EstadoBono
+getTorneo(): TorneoInterno
+estaDisponible(): boolean
+buscar(String): BonoDescuento
+limpiar(): void
SaldoFidelidad
-puntos: int
+SaldoFidelidad()
+getPuntos(): int
+agregarPuntos(int): void
+redimirPuntos(int): void
+redimir(int): void
Text
saldoFidelidad
CodigoDescuento
-codigo: String
DESCUENTO_PROPIO: double = 0.15
DESCUENTO_COMPARTIDO: double = 0.08
+CodigoDescuento(String, Empleado)
+getCodigo(): String
+getPropietario(): Empleado
+porcentajePara(Usuario): double
VentaCafeteria
IMPUESTO_CONSUMO: double = 0.08
PROPINA_SUGERIDA: double = 0.10
-propinaPorcentaje: double
-avisoAlergenosRealizado: boolean
-instalacion: Instalacion
+VentaCafeteria(Usuario)
+VentaCafeteria(Usuario, Instalacion)
+VentaCafeteria(Usuario, LocalDateTime, List, double, double, double, double, int, int, boolean, double, boolean)
+informarAlergenos(): List
+calcularImpuesto(): double
+ajustarPropina(double): void
+liquidar(): double
+getPropinaPorcentaje(): double
+isAvisoAlergenosRealizado(): boolean
+getPropina(): double
-validarBebidaCaliente(): void
VentaTienda
IVA: double = 0.19
+VentaTienda(Usuario)
+VentaTienda(Usuario, LocalDateTime, List, double, double, double, double, int, int, boolean)
+aplicarCodigo(CodigoDescuento): double
+calcularIVA(): double
+liquidar(): double
<Enumeration> EstadoSugerencia
PENDIENTE
APROBADA
RECHAZADA
+values(): EstadoSugerencia[]
+valueOf(String): EstadoSugerencia
-EstadoSugerencia()
<Enumeration> TipoSugerencia
ARTICULO
SNACK
BEBIDA
+values(): TipoSugerencia[]
+valueOf(String): TipoSugerencia
-TipoSugerencia()
SugerenciaProducto
-nombre: String
-descripcion: String
-tipo: TipoSugerencia
-estado: EstadoSugerencia
+SugerenciaProducto(String, String, TipoSugerencia, Empleado)
+getNombre(): String
+getDescripcion(): String
+getEstado(): EstadoSugerencia
+getSolicitante(): Empleado
+aprobar(Administrador): void
+noAprobar(Administrador): void
solicitante: Empleado
club.reportes
ConsultasAdministrativas
+ConsultasAdministrativas()
+consultarReservas(Usuario): List
+consultarPartidos(Usuario): List
+consultarSesiones(Usuario): List
+consolidarVentas(Usuario, GestorTienda, RubroVenta, PeriodoReporte, LocalDate): double
+generarReporteDeportivo(Usuario): ReporteDeportivo
-validarAdministrador(Usuario): void
ReporteDeportivo
-asistencias: List
-resultados: List
-rankings: List
+ReporteDeportivo(List, List, List)
+getAsistencias(): List
+getResultados(): List
+getRankings(): List
<<Enumeration>> PeriodoReporte
DIARIO
SEMANAL
MENSUAL
+values(): PeriodoReporte[]
+valueOf(String): PeriodoReporte
-PeriodoReporte()
<> RubroVenta
TIENDA
CAFETERIA
+values(): RubroVenta[]
+valueOf(String): RubroVenta
-RubroVenta()